

Aula 9 — Laboratório

Comparação Honesta de Todos os Modelos Supervisionados

Objetivos desta sessão

- Aplicar **um único protocolo de validação cruzada** a todos os modelos supervisionados vistos: regressão linear (simples, múltipla e polinomial), k-NN, árvore de decisão, SVR, regressão logística, Random Forest, Naive Bayes e SVM.
- Avaliar **modelos de regressão com validação cruzada**, reportando MAE, RMSE e R^2 como média e desvio.
- Reconhecer duas armadilhas práticas: o **sinal negativo** das métricas de erro no scikit-learn e as **dobras sem embaralhar** em arquivos ordenados.
- Explicar por que comparar modelos com hiperparâmetros "de fábrica" é injusto e **ajustar cada modelo dentro do treino** com [GridSearchCV](#).
- Entender o **otimismo do `best_score_`** e medir o desempenho honesto com **validação cruzada aninhada**.
- Fechar uma escolha de modelo com o **protocolo completo** e usar a **parcimônia** para decidir entre modelos empatados.

1. Onde estamos e o que muda hoje

Até aqui cada família de modelos apareceu em um momento diferente: a regressão foi avaliada com uma divisão treino/teste e as métricas MAE, RMSE e R^2 ; os classificadores foram comparados primeiro numa divisão única e depois com validação cruzada. Faltava juntar tudo. Esta sessão coloca **todos os modelos supervisionados do semestre** sob as mesmas regras e responde à pergunta que qualquer projeto real faz no fim: *qual modelo eu uso, e quanto posso prometer que ele acerta?*

Não há algoritmo novo. O que é novo é o **protocolo**: um conjunto de regras que, se seguido, torna a comparação justa e o número final confiável.

1.1. As regras do protocolo

Regra	Por quê
Mesmos dados e mesmas dobras para todos os modelos (o mesmo objeto KFold/StratifiedKFold)	a diferença entre modelos passa a vir só do algoritmo, e a comparação dobra a dobra fica pareada
Todo pré-processamento dentro do Pipeline	imputação, padronização e one-hot são aprendidos só com o treino de cada dobra: sem vazamento
Um baseline na tabela (média, ou classe mais comum)	é o piso: um modelo que não o supera com folga não aprendeu nada útil
Várias métricas , sempre como média $\pm$ desvio	a média estima o desempenho; o desvio mostra quanto ele varia de uma dobra para outra
Hiperparâmetros escolhidos dentro do treino para todos os modelos	comparar um modelo ajustado com outro "de fábrica" mede o esforço de quem ajustou, não o algoritmo
O teste é usado uma única vez, no fim	se o teste influencia alguma decisão, a nota final fica otimista

2. Regressão: todos os modelos nos aluguéis

Voltamos ao arquivo `alugueis_tratado.csv`: 1000 anúncios, três atributos (`area_m2`, `quartos`, `distancia_km`) e o alvo `valor_aluguel`. Nas aulas de regressão avaliamos os modelos com uma divisão treino/teste. Agora cada modelo passa pelas mesmas 5 dobras, e `cross_validate` calcula três métricas de uma vez:

```
dobras = KFold(n_splits=5, shuffle=True, random_state=42) # regressão: KFold simples
metricas = {"MAE": "neg_mean_absolute_error",
            "RMSE": "neg_root_mean_squared_error",
            "R2": "r2"}
res = cross_validate(modelo, X, y, cv=dobras, scoring=metricas)
mae = -res["test_MAE"] # atenção ao sinal (Seção 2.2)
print(f"MAE = {mae.mean():.1f} ± {mae.std():.1f}")
```

Os modelos entram no dicionário já com o pré-processamento que cada um exige: k-NN e SVR recebem `StandardScaler` (usam distância); a polinomial é `PolynomialFeatures` seguido de `LinearRegression`; a linear simples usa só `area_m2`.

Modelo	MAE (R\$)	RMSE (R\$)	Desvio RMSE	R ²
Polinomial (grau 2)	117,6	146,2	5,2	0,900
SVR (rbf, C=1000, ε=50)	115,5	146,2	3,1	0,900
k-NN (k=15)	123,4	155,7	4,5	0,887
Linear múltipla	126,1	156,7	3,9	0,886
Árvore (max_depth=5)	148,3	188,7	4,6	0,834
Linear simples (só área)	177,6	221,4	7,5	0,771
SVR (rbf, C=1, padrão)	304,1	393,6	16,9	0,281
Baseline (prevê a média)	375,6	465,4	16,6	-0,006

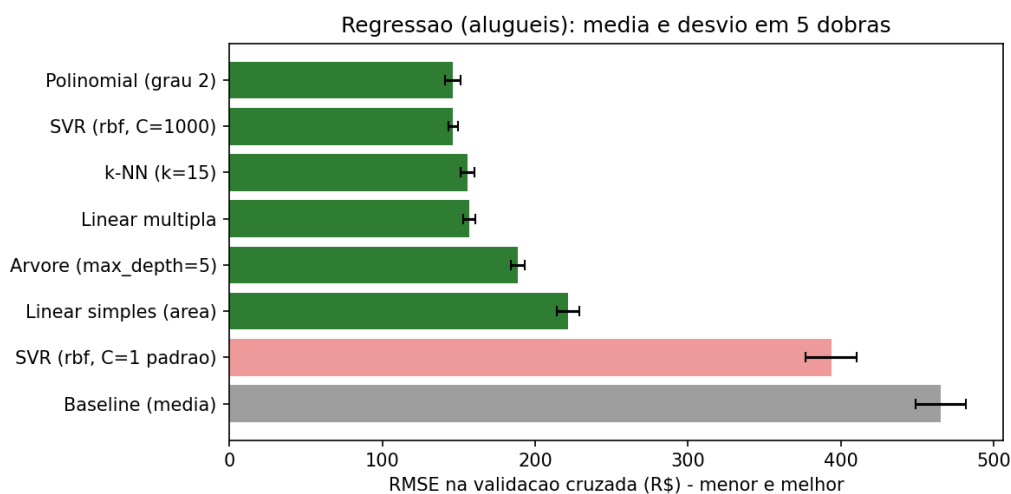


Figura 1 — RMSE de cada modelo de regressão na validação cruzada (média e desvio em 5 dobras). Em vermelho, o SVR com o hiperparâmetro padrão; em cinza, o baseline.

Como ler a tabela

- **Topo empatado.** Polinomial grau 2 e SVR têm RMSE praticamente idêntico (146,2) — a diferença é muito menor que o desvio entre as dobras. Os dois acompanham a leve curvatura que a reta não captura.

- **Segundo grupo.** k-NN e linear múltipla ficam cerca de R\$ 10 atrás. A distância é maior que o desvio (3 a 5), então a diferença é real, mas pequena em reais: vale a pena pagar a complexidade extra? Isso é uma decisão de projeto, não da tabela.
- **Atributos importam mais que o algoritmo.** A mesma regressão linear cai de R^2 0,886 (três atributos) para 0,771 (só área). Nenhum algoritmo recupera informação que não entrou no X.
- **Hiperparâmetros importam muito.** O mesmo SVR vai do penúltimo ao primeiro lugar só trocando C de 1 para 1000. Com o alvo na casa dos milhares de reais, C=1 penaliza tão pouco os erros que o modelo fica quase plano. Guarde este exemplo para a Seção 4.

2.1. MAE, RMSE e R^2 — retomada rápida

Métrica	O que mede	Unidade	Leitura
MAE	média do valor absoluto dos resíduos	a do alvo (R\$)	erro típico; todos os erros pesam proporcionalmente
RMSE	raiz da média dos resíduos ao quadrado	a do alvo (R\$)	RMSE $\geq$ MAE sempre; a diferença cresce quando há erros grandes
R^2	fração da variância do alvo explicada	adimensional	0 = prever a média; negativo = pior que prever a média

2.2. Armadilha 1: o sinal negativo

O scikit-learn trata toda métrica como uma **nota a ser maximizada**. Como erro menor é melhor, MAE e RMSE são devolvidos com o sinal trocado — daí o prefixo **neg_**:

```
cross_val_score(LinearRegression(), X, y, cv=dobras, scoring="neg_mean_absolute_error")
# [-117.3 -128.0 -127.5 -123.1 -134.8] <- MAE NEGATIVO
# reporte: MAE = 126.1 (troque o sinal antes de imprimir ou comparar)
```

Esquecer o sinal leva a conclusões invertidas: o modelo com o "maior" MAE negativo (-117) é o de **menor** erro. O mesmo vale para `GridSearchCV(scoring="neg_root_mean_squared_error")`: o **best_score_** também vem negativo.

2.3. Armadilha 2: arquivo ordenado e dobras sem embaralhar

Muitos arquivos chegam ordenados — por data, por código, por preço. `KFold(n_splits=5)` sem `shuffle=True` corta o arquivo em cinco fatias na ordem em que as linhas aparecem. Se o arquivo estiver ordenado pelo alvo, cada dobra de validação contém uma faixa de preços que o treino quase não viu:

Mesmos dados, ordenados por valor_aluguel	R^2 por dobra	R^2 médio
<code>KFold(n_splits=5)</code> — sem embaralhar	-1,15 -1,01 -3,77 -3,54 -0,25	-1,944
<code>KFold(n_splits=5, shuffle=True, random_state=42)</code>	0,89 0,91 0,87 0,87 0,88	0,886

O modelo é o mesmo; só a divisão mudou. Sem embaralhar, a validação pergunta "você consegue prever os aluguéis mais caros tendo treinado só com os mais baratos?" — uma extrapolação, e o R^2 negativo diz que prever a média seria melhor. **Regra:** use sempre `shuffle=True` com `random_state` fixo. A exceção são dados **temporais**, em que embaralhar é o erro (o futuro vazaria para o passado); para eles existe o `TimeSeriesSplit`, que treina sempre com o passado e valida com o futuro.

3. Classificação: seis famílias e o baseline

No problema das solicitações de empréstimo (450 linhas, alvo **aprovado**) já comparamos cinco classificadores. Hoje entra também a **Random Forest**, e usamos a **validação cruzada repetida**: 5 dobras estratificadas, repetidas 5 vezes com embaralhamentos diferentes — 25 avaliações por modelo, todas com as mesmas divisões. Repetir reduz a dependência de um único sorteio das dobras.

```
dobras = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=42)
res = cross_validate(com_prep(modelo), X, y, cv=dobras,
                    scoring=["accuracy", "f1", "recall"])
```

Modelo (configuração de fábrica)	Acurácia	Desvio	F1	Recall	Melhor em
SVM (rbf)	0,852	0,037	0,848	0,858	12 de 25
Regressão logística	0,851	0,040	0,846	0,849	9 de 25
Random Forest (200 árvores)	0,849	0,038	0,845	0,850	6 de 25
Naive Bayes	0,825	0,044	0,819	0,828	1 de 25
Árvore (max_depth=4)	0,824	0,033	0,813	0,798	1 de 25
k-NN (k=5)	0,798	0,035	0,790	0,791	2 de 25
Baseline (classe mais comum)	0,520	0,004	0,000	0,000	—

A coluna "melhor em" conta em quantas das 25 divisões cada modelo teve a maior acurácia (empates contam para todos). Ela só pode ser calculada porque todos usaram **as mesmas divisões**.

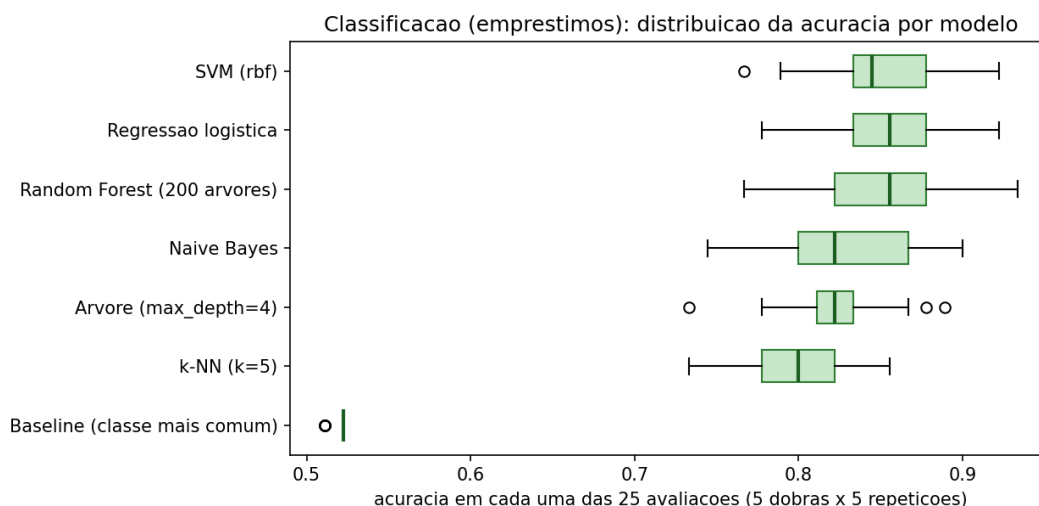


Figura 2 — distribuição das 25 acurácias de cada modelo. As caixas do SVM, da regressão logística e da Random Forest praticamente se sobrepõem.

Como ler

- **Grupo da frente empatado.** SVM, regressão logística e Random Forest diferem em no máximo 0,003 de acurácia, contra um desvio de cerca de 0,04. Nenhum deles "vence"; o SVM é o melhor com mais frequência, mas perde em mais da metade das divisões.
- **Grupo intermediário.** Naive Bayes e a árvore rasa ficam cerca de 0,03 atrás — diferença próxima de um desvio, e confirmada pelas poucas vitórias.
- **O k-NN ficou por último — mas com k=5.** Esse k foi escolhido sem nenhum critério. Antes de concluir que "o k-NN é pior", precisamos dar a ele (e a todos) a chance de escolher seus hiperparâmetros.

4. Ajuste justo: todo modelo merece escolher seus hiperparâmetros

Uma comparação entre modelos com hiperparâmetros "de fábrica" compara, na verdade, **o quanto os valores padrão da biblioteca combinam com este problema**. O SVR da Seção 2 mostrou o tamanho do estrago: com **C=1** ele parecia inútil. O justo é que cada modelo tenha seus hiperparâmetros escolhidos pelo mesmo procedimento — uma busca em grade com validação cruzada, **GridSearchCV**, rodada **só com dados de treino**:

```
grade = {"modelo__n_neighbors": [5, 15, 25, 45, 75, 125]} # nome_do_passo__parametro
busca = GridSearchCV(com_prep(KNeighborsClassifier()), grade, cv=interna)
busca.fit(X, y)
print(busca.best_params_) # {'modelo__n_neighbors': 125}
print(busca.best_score_) # 0.853 <- cuidado: esta nota é otimista
```

4.1. Por que o `best_score_` é otimista

O `best_score_` é a nota da **melhor** configuração, medida nas **mesmas dobras** que serviram para escolhê-la. É como um aluno que faz dez simulados e reporta só a maior nota: parte dela é mérito, parte é sorte daquele simulado. Quanto maior a grade (mais configurações disputando) e menor o conjunto de dados (notas mais ruidosas), maior esse otimismo.

A saída é separar quem **escolhe** de quem **mede**, também dentro da validação cruzada. Na **validação cruzada aninhada**, cada dobra externa guarda sua parte de validação e entrega o restante para uma validação cruzada interna, que escolhe o hiperparâmetro; a configuração escolhida é então medida na dobra externa, que não participou da escolha:

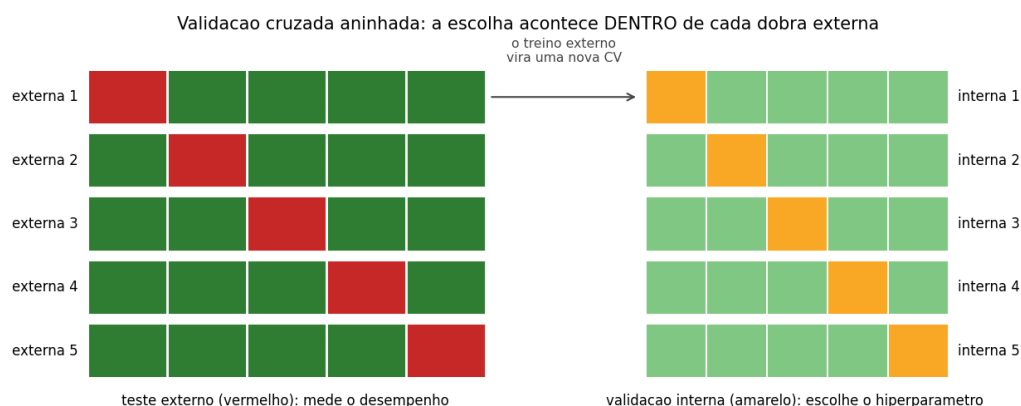


Figura 3 — validação cruzada aninhada. À esquerda, as 5 dobras externas; à direita, o que acontece dentro do treino da dobra externa 1: uma nova validação cruzada só para escolher o hiperparâmetro.

```
busca = GridSearchCV(com_prep(modelo), grade, cv=interna) # quem ESCOLHE
notas = cross_val_score(busca, X, y, cv=externa) # quem MEDE
print(notas.mean()) # estimativa honesta do procedimento "ajustar + treinar"
```

Passar um **GridSearchCV** para o `cross_val_score` é tudo o que é preciso: como o **GridSearchCV** também tem `fit` e `predict`, ele é tratado como um modelo qualquer, e a busca inteira é refeita dentro de cada dobra externa.

Modelo	Configuração escolhida	best_score_	Aninhada	Otimismo
SVM (rbf)	C=100, gamma=0,01	0,876	0,864 ± 0,036	0,011
Random Forest	max_depth=3, min_samples_leaf=5	0,860	0,853 ± 0,029	0,007
k-NN	n_neighbors=125	0,853	0,847 ± 0,041	0,007
Regressão logística	C=0,1	0,858	0,840 ± 0,032	0,018
Árvore	max_depth=3	0,836	0,829 ± 0,033	0,007

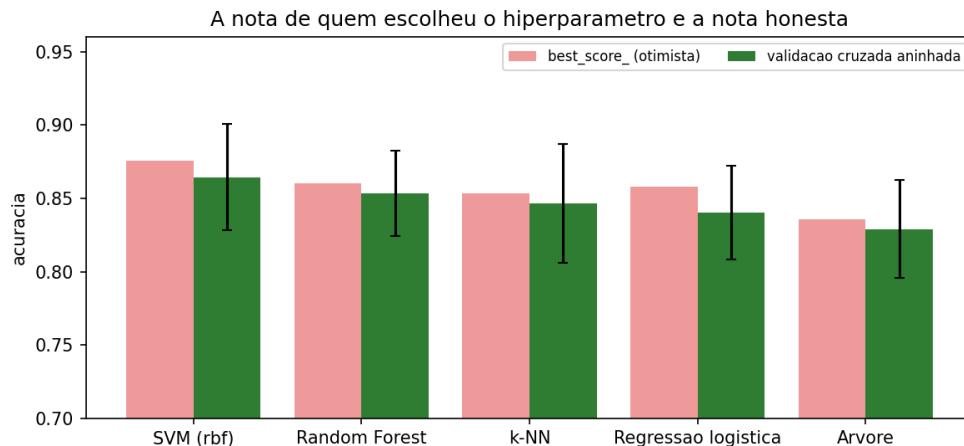


Figura 4 — `best_score_` (rosa) sempre acima da nota aninhada (verde). As barras de erro mostram que, depois do ajuste, os cinco modelos ficam dentro de um desvio uns dos outros.

Como ler

- **O ranking mudou.** O k-NN saiu de 0,798 (k=5) para 0,847 (k=125) e alcançou o grupo da frente. A conclusão da Seção 3 ("o k-NN é o pior") era sobre o k=5, não sobre o k-NN.
- **O otimismo existe e tem sempre o mesmo sinal.** Aqui ele é pequeno (0,007 a 0,018) porque as grades são pequenas; com grades de centenas de combinações ou com poucos dados, ele cresce. O número a reportar é o da validação aninhada.
- **Depois do ajuste, todos estão perto.** A diferença entre o primeiro (0,864) e o último (0,829) é de cerca de um desvio. Neste problema o sinal está nos dados — score de crédito, renda e tempo de emprego — e quase qualquer família bem ajustada o encontra.

5. O protocolo completo: escolher no treino, conferir no teste uma vez

A validação aninhada responde "quanto esse **procedimento** acerta?". Para entregar **um** modelo, usamos o fluxo completo com os três papéis dos dados:

1. Separe o **teste** logo no início (aqui, 25% estratificado) e não olhe para ele.
2. Para cada família, rode o **GridSearchCV** **só no treino**.
3. Escolha a família pela nota de validação cruzada do treino.
4. O **best_estimator_** já vem re-treinado com o treino inteiro, usando a melhor configuração.
5. Meça no teste **uma única vez** e reporte esse número.

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
                                         random_state=42, stratify=y)
for nome, (modelo, grade) in grades.items():
    buscas[nome] = GridSearchCV(com_prep(modelo), grade, cv=interna).fit(X_tr, y_tr)
escolhido = max(buscas, key=lambda n: buscas[n].best_score_)
final = buscas[escolhido].best_estimator_
print(accuracy_score(y_te, final.predict(X_te))) # o teste, pela primeira e última vez
```

Família	Melhor configuração (no treino)	CV no treino
SVM (rbf)	C=10, gamma=0,01	0,867
k-NN	n_neighbors=75	0,852
Random Forest	max_depth=3	0,852

Família	Melhor configuração (no treino)	CV no treino
Regressão logística	C=10	0,846
Árvore	max_depth=4	0,825
Naive Bayes	(sem hiperparâmetro na grade)	0,816

Escolhido o SVM, o teste — 113 solicitações que não influenciaram nenhuma decisão — dá **acurácia 0,867, F1 0,870 e recall 0,926**. O número é coerente com a estimativa do treino (0,867), e é esse que vai para o relatório.

E se o teste decepcionar?

Não volte para trocar de modelo e testar de novo no mesmo teste: na segunda tentativa ele já virou validação, e a nota deixa de ser honesta. Se a nota do teste ficar muito abaixo da validação, investigue a causa (vazamento, divisão estranha, dados diferentes) e, se for preciso recomeçar, reporte que o teste foi reutilizado — ou separe um teste novo.

6. Parcimônia: quando há empate, o custo decide

Vimos várias vezes o mesmo desfecho: três ou quatro modelos empatados dentro do desvio. A tabela de métricas não decide; outros critérios decidem:

Critério	Regressão logística	SVM (rbf)	Random Forest
Acurácia (25 avaliações)	0,851	0,852	0,849
Tempo de treino por dobra	≈ 10 ms	≈ 10 ms	≈ 200 ms
Explicar uma decisão	fácil: um coeficiente por atributo	difícil: vetores de suporte e kernel	médio: importância de atributos, mas 200 árvores
Hiperparâmetros a ajustar	1 (C)	2 (C, gamma)	vários

Em crédito, a instituição precisa explicar ao cliente por que o pedido foi negado. Com desempenho empatado, a regressão logística — rápida, com um coeficiente legível por atributo — é a escolha prática mais defensável. Esse é o **princípio da parcimônia**: entre modelos com desempenho equivalente, prefira o mais simples. Se a diferença de desempenho fosse clara e grande, o raciocínio seria outro.

7. Quadro-resumo dos modelos supervisionados

Os modelos vistos até aqui, lado a lado. Use o quadro para revisar e para decidir por onde começar num problema novo:

Modelo	Tarefa	Ideia central	Hiperparâmetros principais	Precisa padronizar?	Interpretável?
Regressão linear (simples/múltipla)	regressão	combinação linear ajustada por mínimos quadrados	— (os atributos que entram)	não (ajuda a comparar coeficientes)	sim
Regressão polinomial	regressão	regressão linear sobre x , x^2 , ...	grau	recomendado em grau alto	razoável
SVR	regressão	tubo de largura 2ϵ ; só erros fora do tubo contam	C, epsilon, kernel, gamma	sim	pouco
k-NN	ambas	média ou voto dos k vizinhos mais próximos	k	sim	pelos vizinhos
Árvore de decisão	ambas	perguntas "atributo > valor?" em sequência	max_depth, min_samples_leaf	não	sim (se rasa)

Modelo	Tarefa	Ideia central	Hiperparâmetros principais	Precisa padronizar?	Interpretável?
Random Forest	ambas	muitas árvores com amostras diferentes, votando	n_estimators, max_depth, min_samples_leaf	não	parcial
Regressão logística	classificação	combinação linear passada pela sigmoide	C (regularização)	recomendado	sim
Naive Bayes	classificação	teorema de Bayes com atributos independentes	— (quase nenhum)	não	razoável
SVM	classificação	fronteira de margem máxima; kernel para curvas	C, kernel, gamma	sim	pouco

Sobre o hiperparâmetro **C**: nos três modelos em que aparece (SVR, SVM e regressão logística), C alto significa **menos regularização** — o modelo persegue mais de perto os dados de treino (mais risco de sobreajuste); C baixo significa um modelo mais contido (mais risco de subajuste).

8. Checklist de uma comparação honesta

- Todos os modelos usaram **o mesmo objeto de dobras**, com `shuffle=True` e `random_state` fixo (ou `TimeSeriesSplit`, se os dados forem temporais)?
- O pré-processamento está **dentro do Pipeline** de cada modelo?
- Há um **baseline** na tabela, e todos o superam com folga?
- As métricas aparecem como **média ± desvio**, e as de erro estão com o sinal corrigido?
- **Todos** os modelos tiveram seus hiperparâmetros escolhidos pelo mesmo procedimento, dentro do treino?
- O desempenho reportado vem da **validação aninhada** ou de um **teste usado uma única vez** — e não do `best_score_`?
- Diferenças menores que o desvio foram tratadas como **empate**, e o empate foi decidido por **parcimônia** e custo?

O que guardar desta sessão

- Um protocolo único — mesmas dobras, Pipeline, baseline, média ± desvio — vale para regressão e para classificação.
- Na regressão, `cross_validate` com MAE, RMSE e R^2 ; os erros voltam negativos (`neg_`) e precisam ter o sinal trocado.
- Arquivo ordenado + dobras sem embaralhar = validação que extrapola e dá R^2 negativo. Use `shuffle=True` (ou `TimeSeriesSplit` em dados temporais).
- Comparar modelos "de fábrica" é injusto: o mesmo SVR vai de último a primeiro só trocando **C**, e o k-NN sobe de 0,798 para 0,847 com k ajustado.
- O `best_score_` é otimista porque escolhe e mede nas mesmas dobras; a validação cruzada aninhada separa quem escolhe de quem mede.
- Protocolo completo: teste separado no início, `GridSearchCV` só no treino, teste usado uma única vez.
- Empate dentro do desvio se decide por parcimônia: o modelo mais simples, rápido e explicável.

Arquivos desta sessão

- `lab09_demo.py` — demonstração guiada (6 blocos); gera as quatro figuras desta apostila. Leva cerca de 2 minutos, por causa da validação aninhada.
- `lab09_comparacao_honesta.ipynb` — a mesma demonstração em notebook, com explicações entre as células.
- `lab09_exercicios.py` — mini-laboratório do aluno (preencha os `# TODO` e responda o RELATORIO).

- `alugueis_tratado.csv` — dados de aluguel usados na parte de regressão (deixe na mesma pasta dos scripts).

Mini-laboratório (10,0 pontos — código 40% | relatório 60%)

Exercício	Tema	Pontos
1	Regressão: sete modelos com 10 dobras, MAE/RMSE/ R^2 e o RMSE de treino	2,0
2	Armadilha do arquivo ordenado: k-NN com e sem embaralhar as dobras	1,5
3	Classificação: seis famílias, 5 dobras $\times$ 3 repetições, comparação pareada com a regressão logística	2,0
4	Ajuste justo: configuração de fábrica $\times$ GridSearchCV $\times$ validação aninhada	2,5
5	Protocolo completo na regressão: escolher no treino, testar uma vez	2,0